

TensorMLD: A Tensorized and Compressed qLDPC Maximum-Likelihood Decoder for Accurate and Fast Detection of the Most Likely Logical Error

Liqiang Lu
Zhejiang University
Hangzhou, China
liqianglu@zju.edu.cn

Debin Xiang*
Zhejiang University
Hangzhou, China
db.xiang@zju.edu.cn

Kaiwen Zhou
Zhejiang University
Hangzhou, China
kaiwenzhou@zju.edu.cn

Boyu Jin
Zhejiang University
Hangzhou, China
boyujin@zju.edu.cn

Chenning Tao
Zhejiang University
Hangzhou, China
tcn@zju.edu.cn

Wuwei Tian
Zhejiang University
Hangzhou, China
sonder@zju.edu.cn

Anbang Wu
Shanghai Jiao Tong University
Shanghai, China
anbangwu@sjtu.edu.cn

Fangxin Liu
Shanghai Jiao Tong University
Shanghai, China
liufangxin@sjtu.edu.cn

Jingwen Leng
Shanghai Jiao Tong University
Shanghai, China
leng-jw@cs.sjtu.edu.cn

Minyi Guo
Shanghai Jiao Tong University
Shanghai, China
guo-my@cs.sjtu.edu.cn

Jianwei Yin*
Zhejiang University
Hangzhou, China
zjuyjw@cs.zju.edu.cn

Abstract—Quantum error correction is vital for building reliable quantum computers, and quantum Low Density Parity Check (qLDPC) codes are highly promising due to their efficient encoding. However, current decoders often struggle with low decoding accuracy. This is because they focus on identifying physical errors, which cannot always correct the underlying logical fault. While maximum likelihood decoding offers optimal performance by identifying the most likely logical error, its application to qLDPC codes is computationally prohibitive, largely due to the high-dimensional encoding between the logical and physical error spaces. We introduce TensorMLD, an efficient MLD decoder that overcomes this barrier. Our key innovation maps decoding into a spin glass model computable via tensor networks. We then compress this representation through tensor decomposition and offline compression, reducing peak memory requirements by up to $10^{16} \times$ and computational workload by up to $10^{17} \times$. For practical decoding, we develop a parallel marginal probability strategy equipped with GPU and FPGA accelerators that enables efficient decoding with the compressed tensors. Experiments demonstrate TensorMLD achieves an average $9.07 \times$ lower logical error rate than the state-of-the-art RelayBP decoder while maintaining comparable latency.

Index Terms—Quantum error correction, QLDPC codes, Maximum likelihood decoding, Tensor networks, Tensor compression

I. INTRODUCTION

Quantum error correction (QEC) is essential for building reliable and large-scale quantum computers [47]. QEC protects

quantum information against noise by redundantly encoding logical qubits into multiple physical qubits [1], [10]. Quantum low-density parity-check (qLDPC) codes are a promising code. Unlike the commonly used surface code [7], [19], which needs a quadratic number of physical qubits to encode logical qubits, qLDPC codes achieve a constant encoding rate and thus scale linearly in physical qubits. This high encoding rate makes qLDPC codes attractive for scalable fault-tolerant architectures. Recent work has further demonstrated the feasibility of implementing qLDPC codes on neutral-atom and superconducting platforms [8], [51], [59].

Beyond code design, the decoder is central to the QEC workflow [2], [14], [49], [56], since it identifies both the type and location of errors. A QEC cycle begins by encoding a logical qubit into an entangled state of many physical qubits [28], [31]. Repeated stabilizer measurements then extract syndromes (binary string) that capture the presence and location of physical errors without collapsing the logical state. The decoder interprets these syndromes to infer the underlying error configuration and determines the appropriate correction operator. As a result, decoder performance directly dictates the fault-tolerance capability of the entire quantum system.

The design of a qLDPC decoder depends heavily on the target error model, which is typically categorized into *the most likely physical error* and *the most likely logical error*. The former is addressed by minimum-weight decoding (MWD), which selects the physical error with the lowest weight that is consistent with the syndrome. As shown in Figure 1 (a), MWD returns only a single most probable physical error ($\mathbf{X}_2$). In contrast, maximum-likelihood decoding (MLD) evaluates

* Corresponding author.

This work was supported by the Zhejiang Provincial Pioneer (Jianbing) Project (No. 2026C01006), the Zhejiang Provincial Natural Science Foundation of China for Distinguished Young Scholars (No. LR25F020002), and the Fundamental Research Funds for the Central Universities (No. 226-2026-00075).

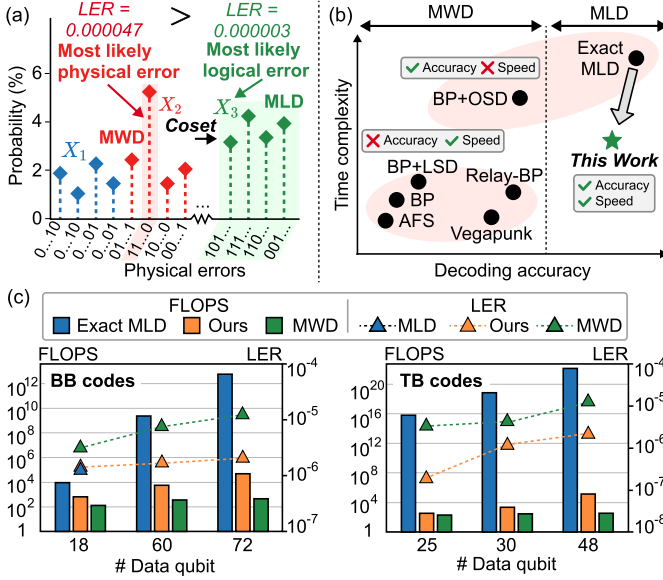


Fig. 1. (a) Difference between minimum weight decoding (MWD) and maximum likelihood decoding (MLD). (b) Trade-off between decoding accuracy and time complexity of existing QEC decoders. They either have low accuracy or high time complexity. (c) Computational cost (FLOPS) and logical error rate (LER) comparison among exact MLD, TensorMLD, and MWD (BP+OSD) decoder. The LER of exactMLD lacks the peak memory requirements have exceeded 80GB.

all physical errors within a coset and identifies the most probable logical error (X_3), achieving globally optimal performance [16], [27], [46]. Due to quantum degeneracy, the true logical error corresponds to a large set of physical errors, making MWD inherently approximate and often inaccurate. In our experiments, even the state-of-the-art BP+OSD decoder [45] correctly compensates the true logical error with only 18.4% probability on a 72-qubit bivariate bicycle (BB) code.

Achieving fault-tolerant quantum computing requires low-latency error-correction feedback, which forces decoders to balance accuracy and computational cost. Decoding qLDPC codes is particularly challenging compared with surface codes [21], [34], [40], [55], [61], as surface codes have regular 2D locality while qLDPC codes feature irregular hypergraphs with non-local stabilizers. Figure 1 (b) illustrates recent progress in improving the performance of decoding qLDPC codes. We can observe that most studies focus on the MWD principle because MWD inherently approximates the decoding problem to finding a single, most likely physical error, which is computationally more tractable than estimating logical error coset probability with MLD. Within the MWD framework, several works aim to achieve higher accuracy [39], [45], while others seek to reduce decoding latency [24], [61], [62]. For example, relay-BP improves the accuracy by running BP under different disordered memory strengths [39], and Vegapunk reduces the latency via guessing the physical error vector in parallel from the decoupled check matrices [62]. On the other hand, studies of the MLD principle mainly concentrate on arithmetical analysis, and the acceleration is less explored. Specifically, these works often focus on developing algebraic

or statistical bounds for the logical error rate [13], [27], [59]. For example, Chubb et.al. give the exact threshold of the surface code under mildly correlated bit-flip noise [13].

Compared to MWD, MLD inherently delivers a higher fault-tolerance threshold [58], allowing for higher noise levels in the future design of large-scale quantum chips. Furthermore, a low logical error rate also helps to simplify the connectivity among qubits [11], [13], making the chip less sensitive to the technology. Mathematically, the MLD calculates the logical probability by accumulating all probabilities of physical errors that belong to the same logical error [27]. And then, it identifies the largest logical probability among all possible logical errors. This enlarges the computation workload of MLD, causing a higher FLOPS compared with MWD, as shown in Figure 1 (c). However, the computation flow of MLD is entirely different from that of MWD, making existing MWD optimizations ineffective for accelerating MLD. For example, applying the early-termination [15] or hardware-pruning [2], [62] MWD accelerator to the MLD decoder will severely damage the fidelity (e.g., logical error rate increases from 10^{-6} to 10^{-4}), due to the inaccurate truncation of the critical probability summation. *In a word, an accuracy-complexity MLD co-design is strongly desired, which can inherit its accuracy advantage while tackling the overwhelming computational pressure.*

In this paper, we propose TensorMLD, the first MLD decoder design that systematically balances the accuracy and time complexity. Our key novelty lies in the tensorization and compression for the decoding process of the most likely logical error. Considering that MLD is globally disordered with 0 or 1 values, we map the decoding problem into a spin-glass model [26], [29], which can then be computed through the tensor network [38], [41]. Although this mapping has been explored in prior theoretical studies [11], [13], [29], they focus on planar codes like repetition codes and surface codes. Directly applying the mapping to qLDPC codes still exhibits 2^n complexity, resulting in a significant computation cost and memory bottleneck. Thus, we then propose an exact tensor decomposition scheme that shrinks the original $O(2^n)$ tensor to a matrix product state (MPS) with a linear complexity of $O(8n)$, e.g., from 3.9GB to 8.7KB for 254-size trivariate bicycle codes. Furthermore, we flatten the high-dimensional connection in MPS into a 1D chain and successfully compress the computation requirement from TFLOPS-level to KFLOPS-level.

Next, we present our online decoding flow and hardware accelerator architecture. While the 1D chain MPS encodes all logical error probability information, directly enumerating all logical errors still incurs an exponential computational cost. To address this issue, we develop a partial tracing strategy to compute the conditional marginal probability of each logical spin in parallel. This approach maintains decoding accuracy while reducing the time complexity to polynomial time related to the number of logical qubits. Finally, we implement this efficient decoding flow on an FPGA device through a dedicated acceleration architecture. We also implemented a

GPU acceleration architecture, which leverages Tensor cores for matrix product operations and CUDA cores for calculating marginal logical probabilities.

Our contributions are summarized as follows:

- Targeting the most likely logical error, TensorMLD is the first work that tries to balance the decoding accuracy and time complexity. Inspired by the spin glass model, we tensorize the decoding flow into the computation of high-rank tensors, exposing the opportunity for parallelism
- We propose two compression techniques: exact tensor decomposition and offline approximated tensor compression, which greatly cut down the computational cost while exhibiting little accuracy loss.
- We propose the overall decoding flow with an efficient hardware accelerator. We employ a conditional marginal probability strategy to achieve polynomial-time complexity, equipped with GPU and FPGA accelerator.

Experimental results demonstrate that TensorMLD achieves the lowest logical error rate, which is $9.07\times$ lower than the state-of-the-art RelayBP decoder. TensorMLD achieves high decoding accuracy while reducing max tensor size by $8.0 \sim 2048.0\times$ and computational workload by $14.2 \sim 10^{17}\times$, compared with the original MLD tensor network decoder. Concretely, TensorMLD compresses the memory footprint from 3.9GB to 8.7KB in 254-size trivariate bicycle codes and reduces the computation workload from 10^7 PFLOPS to 131KFLOPS in 48-size trivariate bicycle codes.

II. BACKGROUND

A. Basics of qLDPC codes

Quantum low-density parity-check (qLDPC) codes are a class of quantum error-correcting codes defined by two sparse binary parity-check matrices H_X and H_Z , which satisfy the commutation constraint $H_X H_Z^T = 0$. These two matrices describe the stabilizer that detects errors, where H_X corresponds to phase (Z) errors and H_Z relates to bit-flip (X) errors, respectively. As shown in Figure 2 (a), the structure of qLDPC codes can be represented by a Tanner graph. It is a bipartite graph that connects data qubits to parity checks, where the edges stem from the non-zero entries in H_X and H_Z . Measuring the parity check qubits will give the syndrome that encodes the error types and locations.

Unlike surface codes, where the decoding problem reduces to finding minimum-weight perfect matchings on a planar graph [49], [57], qLDPC codes generally induce a hypergraph structure due to the complex connectivity and the higher weight of stabilizers [6]. Each stabilizer can involve a logarithmic or constant number of qubits with non-local interactions, resulting in dense and irregular Tanner graphs. This hypergraph topology leads to an NP-hard decoding problem, where the ideal objective is to infer the most probable logical error given a syndrome [25].

B. MLD Decoding

Noise on the qLDPC codes circuit can be described as a physical error vector $\vec{e}$, where $e_j = 1$ means an error occurs in

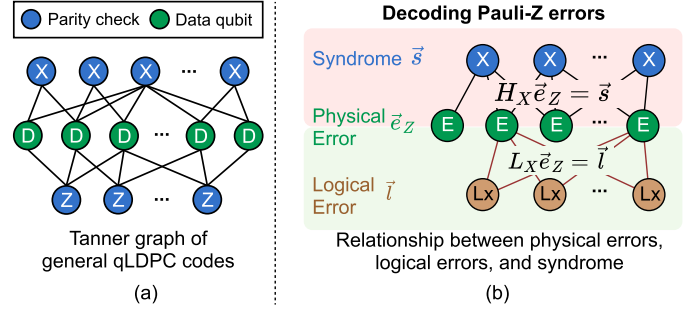


Fig. 2. (a) Tanner graph of general qLDPC codes. (b) The relationship between physical errors, logical errors, and syndrome.

the j -th position, while $e_j = 0$ means no error in this position. The physical errors can be decomposed into Pauli-X errors and Pauli-Z errors, which eventually cause the logical Pauli-X or Pauli-Z errors on logical qubits. Thus, the decoding objective of QLDPC is to infer the logical errors and correct the logical qubits. Since qLDPC codes belong to CSS codes, the Pauli-X and Pauli-Z errors can be decoded independently [45], [52]. Without loss of generality, we focus on the decoding of Pauli-Z errors.

To detect the Pauli-Z errors $\vec{e}_Z$, measurements are applied on the X-type parity check qubits to give the syndrome vectors $\vec{s}$. As shown in Figure 2 (b), the relationship between physical error $\vec{e}_Z$, logical error $\vec{l}$, and syndrome $\vec{s}$ can be expressed as

$$\begin{bmatrix} H_X \\ L_X \end{bmatrix} \vec{e}_Z = \begin{bmatrix} \vec{s} \\ \vec{l} \end{bmatrix} \pmod{2}. \quad (1)$$

Here, H_X is the parity check matrix, where each row is an X-stabilizer. L_X is the logical check matrix, where each row is an X logical operator.

The maximum likelihood decoding (MLD) for qLDPC codes centers on finding the most likely logical error $\vec{l}^*$ given an observed syndrome $\vec{s}$, following Equation (1). Here, we use Pauli-Z errors $\vec{e}_Z = \{e_1, e_2, \dots, e_j, \dots\}$ for illustration, where each error $e_j \in \{0, 1\}$ has physical error probability p_j . The goal is to maximize the logical probability $P(\vec{l}|\vec{s})$ under a given syndrome $\vec{s}$,

$$\vec{l}^* = \arg \max_{\vec{l}} P(\vec{l}|\vec{s}). \quad (2)$$

Here, the logical probability $P(\vec{l}|\vec{s})$ is calculated by accumulating all physical errors $\vec{e}_Z$ that belong to the logical error $\vec{l}$ and match the syndrome $\vec{s}$,

$$P(\vec{l}|\vec{s}) = \sum_{\{\vec{e}_Z | H_X \vec{e}_Z = \vec{s}, L_X \vec{e}_Z = \vec{l}\}} P(\vec{e}_Z) \quad (3)$$

$$P(\vec{e}_Z) = e^{-\sum_{j=1}^n 2w_j e_j}, \quad w_j = \ln((1-p_j)/p_j)/2 \quad (4)$$

where $P(\vec{e}_Z)$ is the physical error probability and p_j is the physical probability of occurring j -th error, i.e., $e_j = 1$. w_j is called error weight of j -th error. After detecting the most likely logical errors $\vec{l}^*$, the logical operator for error correction is,

$$\hat{L} = H_{\text{des}}^T \vec{s} \oplus L_X^T \vec{l}^* \quad (5)$$

where H_{des} is the matrix with each row a Z-stabilizer.

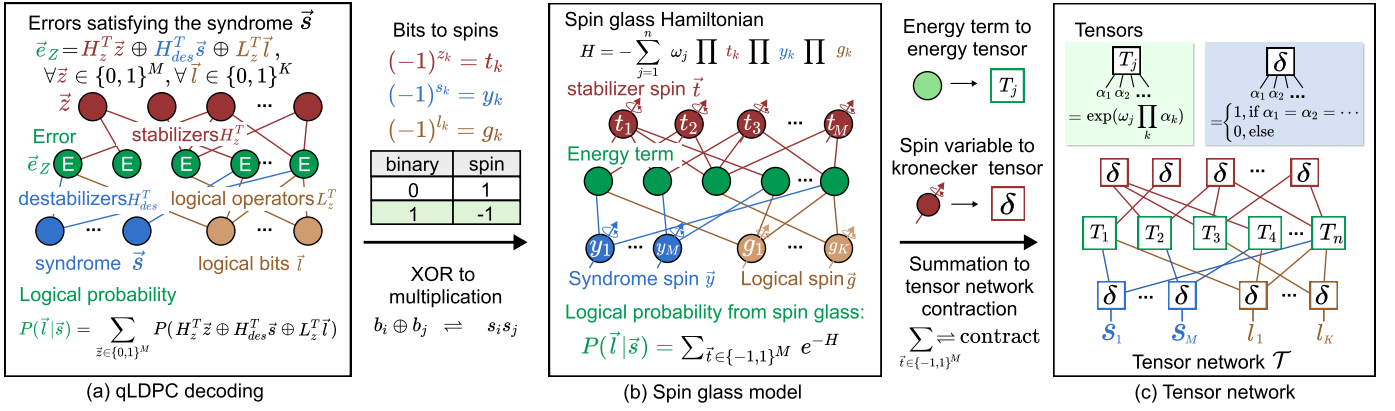


Fig. 3. Formulating the (a) maximum likelihood decoding as (b) minimizing the free energy of the spin glass model, and using (c) the tensor network to calculate the free energy.

III. TENSORMLD TENSORIZATION AND COMPRESSION

To balance the decoding accuracy and computational complexity, we first convert the constrained calculation into an unconstrained summation format. Then, the computation for this summation is tensorized using the spin glass model, where the errors and syndromes are converted into spins. After this, the logical probabilities are calculated by the free energies of the spin glass system, which can be efficiently computed by the tensor network. Although the tensorization exposes the opportunity to parallelize the MLD computation, the overall memory footprint and computational workload (FLOPs) still grow exponentially with the qLDPC code size. Identifying that the tensors from the spin glass model are highly structured, which can be expressed by a product of constant-dimensional matrices, we propose an exact tensor decomposition that shrinks the original $O(2^n)$ tensor to a matrix product state (MPS) with a linear complexity of $O(8n)$, from 3.9GB to 8.7KB for 254-size trivariate bicycle codes. To further reduce the computational workload, we flatten the high-dimensional connection in MPS into a 1D chain. We achieve this by swapping the tensor indices and merging adjacent matrices to eliminate high-dimensional connections. This compression process incurs some approximation error but reduces the computational workload significantly. Finally, we successfully compress the computation requirement from 6.1 TFLOPS to 49 KFLOPS in the 72-size bivariate bicycle code, with little accuracy loss.

A. Tensorizing MLD with Spin Glass Model

Calculating the logical error probability with Equation (3) is non-trivial, as we need to determine the total probability by scanning all physical errors that satisfy two constraints: 1) belong to the same logical error. 2) produce the observed syndrome; Since checking these two constraints will consume computing resources, we tend to convert the formula into an unconstrained summation format. Clearly, as shown in Figure 3 (a), we decompose the logical error space $\vec{e}_Z$ into stabilizer subspaces H_z^T that are shifted by destabilizers H_{des}^T and logical operators L_z^T ,

$$\vec{e}_Z = H_z^T \vec{z} \oplus H_{des}^T \vec{s} \oplus L_z^T \vec{l}, \forall \vec{z} \in \{0,1\}^M \quad (6)$$

Combining this equation with the logical probability $P(\vec{l}|\vec{s})$ in Equation (3) gives an unconstrained summation over all possible stabilizers $\vec{z}$ as

$$P(\vec{l}|\vec{s}) = \sum_{\vec{z} \in \{0,1\}^M} P(H_{des}^T \vec{s} \oplus H_z^T \vec{z} \oplus L_z^T \vec{l}). \quad (7)$$

However, this summation still has exponential complexity. To further simplify this, we recast the decoding problem within the framework of the spin glass model, which has broad application in discrete optimization problems [54]. A representative application is that this model is widely used in statistical mechanics to study systems with disorder and frustrated interactions, matching the properties of our formulation. Concretely, the error in decoding problem has two discrete states, 0 or 1, which resemble the two states in the spin glass model, the up(+1) and down(-1) spin rotations. This inspires us to represent stabilizer bits $\vec{z}$, syndrome $\vec{s}$, and logical bits $\vec{l}$ in Equation (7) as stabilizer spins $\vec{t}$, syndrome spins $\vec{y}$ and logical spins $\vec{g}$ via the bits-to-spin mapping,

$$(-1)^{z_k} = t_k, \quad (-1)^{s_k} = y_k, \quad (-1)^{l_k} = g_k. \quad (8)$$

As shown in Figure 3 (b), mapping bits to spins transforms the probability into the energy of a spin glass determined by the Hamiltonian H ,

$$H = -\sum_{j=1}^n \omega_j \prod_{\{k|H_z(k,j) \neq 0\}} t_k \prod_{\{k|H_{des}(k,j) \neq 0\}} y_k \prod_{\{k|L_z(k,j) \neq 0\}} g_k. \quad (9)$$

The logical error probability under the spin glass system is

$$P(\vec{l}|\vec{s}) = \sum_{\vec{t} \in \{-1,1\}^M} e^{-H} \quad (10)$$

Finally, it has been demonstrated that the spin glass model can be efficiently calculated via a tensor network [11], exposing the opportunity to tensorize Equation (10). By mapping the energy terms and spin variables in Figure 3 (b) to the energy tensor T_j and Kronecker Tensor δ , we convert the summation operation in Equation (10) into the contraction of the tensor network $\mathcal{T}$ in Figure 3 (c). As a result, the logical error probability $P(\vec{l}|\vec{s})$ is reformulated as the contraction of the tensor network $\mathcal{T}$.

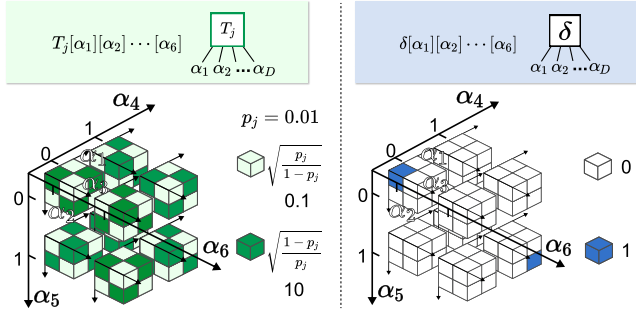


Fig. 4. The numerical example of the tensors used in the tensor network decoder.

Lemma 1: The probability of logical error $\vec{l}$ with given syndrome $\vec{s}$, noted as $P(\vec{l}|\vec{s})$, can be calculated via tensor network $\mathcal{T}$ contraction as

$$P(\vec{l}|\vec{s}) = \frac{\text{contract}(\mathcal{T}(\vec{l}, \vec{s}))}{\text{contract}(\mathcal{T}(\vec{s}))}. \quad (11)$$

This tensor network $\mathcal{T}$ contains energy tensor T_j and Kronecker tensor δ_i , which are defined as

$$T_j[\alpha_1][\alpha_2] \cdots = \exp\left(\omega_j \prod_k (-1)^{\alpha_k}\right) \quad (12)$$

$$\delta_i[\alpha_1][\alpha_2] \cdots = \begin{cases} 1, & \text{if } \alpha_1 = \alpha_2 = \cdots \\ 0, & \text{else} \end{cases} \quad (13)$$

where ω_j is the error weight of the j -th physical error, which is calculated from physical error rate p_j by Equation (4). α_k is the indices of each tensor which has two value $\{0, 1\}$. Furthermore, the Equation (12) can be rewritten as the expression of p_j ,

$$T_j[\alpha_1][\alpha_2] \cdots = \begin{cases} \sqrt{p_j/(1-p_j)}, & \text{if } \sum_k \alpha_k = 1 \bmod 2 \\ \sqrt{(1-p_j)/p_j}, & \text{if } \sum_k \alpha_k = 0 \bmod 2 \end{cases} \quad (14)$$

Figure 4 plots a numerical example of these two types of tensors, which are derived from the 18-size BB code under physical error rate $p_j = 0.01$. For instance, here are two tensor elements.

$$T_j[0][1][1][0][1][0] = \sqrt{p_j/(1-p_j)} \approx 0.1, \\ T_j[1][1][1][1][0][1] = \sqrt{(1-p_j)/p_j} \approx 10$$

For the six-rank Kronecker tensor δ_j , only two elements are one and the rest are zeros.

$$\delta_j[0][0][0][0][0][0] = 1, \\ \delta_j[1][1][1][1][1][1] = 1$$

B. Exact Tensor Decomposition via Matrix Product State

The raw tensor network involves n -rank tensors with 2^n elements, leading to huge computation cost and memory bottleneck. Prior compression techniques often employ singular value decomposition to decompose the tensors into low-rank matrix product states (MPS) or tree tensor network states [33], [42]. However, this coarse-grained decomposition only saves the top largest singular values of tensors and drops the other singular values, causing a significant accuracy loss.

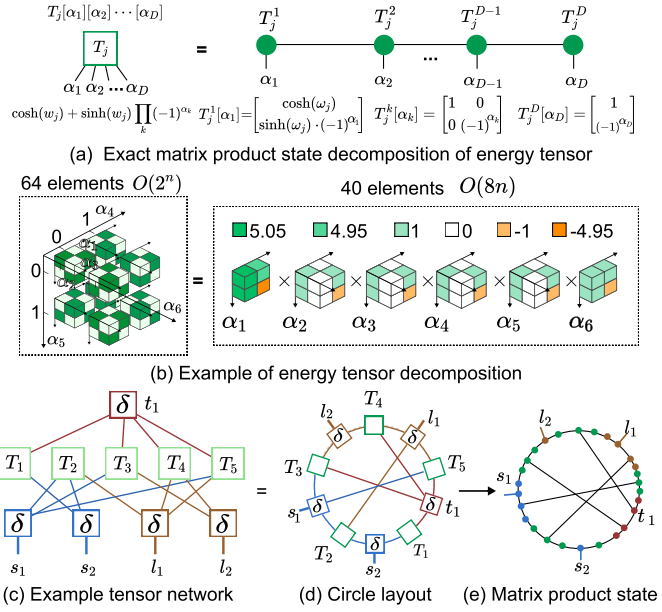


Fig. 5. Decomposing the tensor network into matrix product states.

Considering the binary nature of the errors and check matrices in MLD, we observe that the tensors derived from the spin glass model exhibit a highly structured nature. The pattern includes: 1) each element only has two values (Equation (13) and Equation (14)); 2) the same values are in diagonal distribution or interleaved distribution. Identifying these features, we propose an exact decomposition scheme that can equivalently decompose the tensor into a series of smaller matrices, as shown in Figure 5 (a). Specifically, since the results of spin multiplication only have two values, -1 or 1, the tensor can only be $e^{-\omega_j}$ and e^{ω_j} , transforming the tensor T_j in Equation (12) to hyperbolic functions.

$$T_j = \cosh(\omega_j) + \sinh(\omega_j) \prod_k (-1)^{\alpha_k} \quad (15)$$

We can verify that this tensor T_j can be represented by matrix products with the tensors T_j^k in Figure 5 (a).

$$T_j[\alpha_1][\alpha_2] \cdots [\alpha_D] = \prod_{k=1}^D T_j^k[\alpha_k] \quad (16)$$

Figure 5 (b) gives the example of decomposing a six-rank energy tensor into six three-rank tensors, shrinking the tensor size from 64 elements to 40 elements. For the tensor δ in Equation (13), the elements that share the same index α_k have the same values. Thus, it can be exactly decomposed into low-rank tensors δ .

$$\delta[\alpha_1] \cdots [\alpha_D] = \sum_{\vec{\beta}} \delta[\alpha_1][\beta_1] \delta[\beta_1][\alpha_2][\beta_2] \cdots \delta[\beta_{D-1}][\alpha_D]. \quad (17)$$

Figure 5 (c)-(e) visualizes the tensorization process of the entire tensor network, where the tensor network is derived from Figure 3 (c). To better understand dimension reduction, Figure 5 (d) organizes it in a circular layout, maintaining the original connectivity. In this circle, we can observe that each tensor is connected with four neighboring tensors at most. By decomposing it into matrix product states, the tensors are

converted into three-rank tensors or two-rank tensors, and each has at most three neighbors, as depicted in Figure 5 (e).

C. Tensor Network Compression by Offline Contraction

Although MPS partially addresses the computational pressure, especially the memory bottleneck, there remain tremendous computations arising from the high-dimensional connections among matrices. A natural idea is to compress the connectivity offline and introduce a low-dimensional topology MPS when online decoding. Observing that the high-dimensional connection basically results from the internal edges in the circle, as shown in Figure 5 (d), our intuition is to eliminate these internal edges. To achieve this goal, we propose the *contraction-swap-merge* method, which iteratively contracts the small tensors, swaps the tensor indices to put the internal edges in adjacent positions, and merges these edges. These three operations used in this method are defined as follows.

- The *contract* operation is the basic operation in the tensor network, which merges two adjacent tensors into a new tensor by contracting the edge index between them, as shown in Figure 6 (a). The cost of *contract* operations between two tensors with d_1 dimensions and d_3 dimensions is $O(d_1 d_2 d_3)$, where d_2 is the bond dimension between them (The bond dimension is the dimension of the multiplication between two tensors).
- The *swap* operation is defined as swapping the two adjacent indices in MPS, such as the α_2 and α_3 indices in Figure 6 (b). The *swap* cost differs in two cases. Swapping between pairs like (T, T) , (T, δ) , (δ, δ) has no cost. However, for other cases, swapping between tensors needs to use a *contract* operation and singular value decomposition with cost $O(d^6)$. This can introduce truncation error when the number of nonzero singular-value exceeds χ . The squared truncation error equals the discarded singular-value weight ($\epsilon_k = \sum_{i>\chi} \sigma_i^2$), and the accumulated error over (K) swaps is conservatively bounded by $(\sum_k \epsilon_k)$.
- The *merge* operation uses two *contract* operations to merge the adjacent edges between two MPS into one edge with a higher dimension. The cost of the *merge* operation comes from the *contract* operation, as shown in Figure 6 (c).

The *swap* operation can be approximate when it truncates singular values. Thus, we use a greedy algorithm to reduce lossy swaps, as shown in Algorithm 1. We use a graph $G(V, E)$ to denote this tensor network, the node V representing the tensors and edges E representing the connected indices between tensors $A^{(i)}, A^{(j)}$ with a bond dimensions $D_{i,j}$. The initial bond dimension is 2. Tensors with outer logical and syndrome indices in set P are preserved, forming a 1D matrix product state chain as output. Specifically, we first decompose the tensors into MPS representation using the exact decomposition (Line 1). Then the algorithm loops until all nodes are in the preserved set P (Line 2), stopping at a 1D chain. It selects the edge (i, j) with minimal contraction cost (Line 3), moves

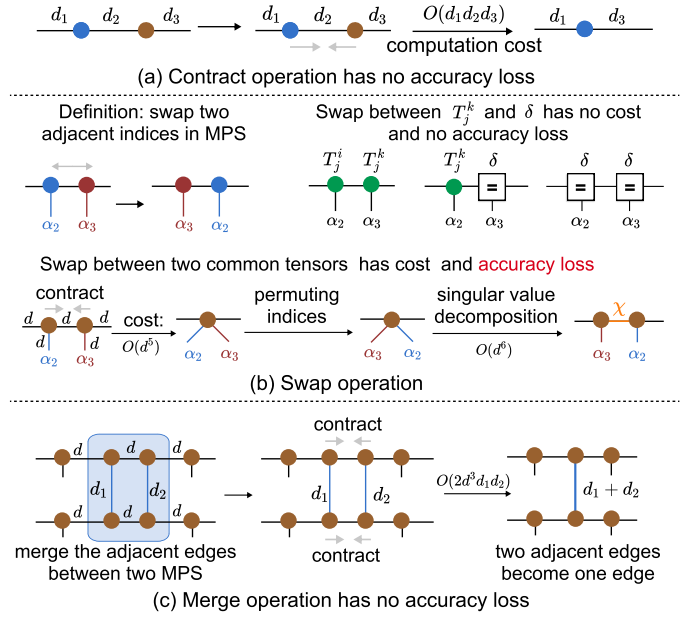


Fig. 6. Operations definition and cost in offline contraction.

Algorithm 1: Contraction-swap-merge Algorithm

Input: Tensor network with tensors $A^{(1)}, \dots, A^{(n)}$, connectivity Tanner graph $G(V, E)$, set of tensors with outer logical and syndrome indices $P \subseteq V$

Output: 1D-chain MPS containing tensors in P

- 1 Exactly decompose every tensor to MPS;
- 2 **while** $\exists v \notin P$ **do**
- 3 Find edge $(i, j) \leftarrow \arg \min_{(\mu, \nu) \in E, \{\mu, \nu\} \not\subseteq P} [\sum_{b \in \partial \mu} \log(D_{b, \mu}) + \sum_{b \in \partial \nu} \log(D_{b, \nu}) - 2 \log(D_{\mu, \nu})]$;
- 4 Move tensor at edge (i, j) in $A^{(i)}$ to MPS tail ;
- 5 Move tensor at edge (i, j) in $A^{(j)}$ to MPS head ;
- 6 Contract edge (i, j) to combine MPS $A^{(i)}$ and $A^{(j)}$ into a new MPS;
- 7 Update $E \leftarrow E \setminus \{(i, j)\}$;
- 8 **for** $k \in \partial j$ **do**
- 9 Remove (j, k) from E ;
- 10 **if** $k \in \partial i$ **then**
- 11 Swap to place duplicate edges (i, k) adjacent in $A^{(i)}$;
- 12 Merge adjacent tensors, combining edges to increase $D_{k, i}$;
- 13 **else**
- 14 Add (i, k) to E ;
- 15 **if** $j \in P$ **then**
- 16 Add i to P ;
- 17 **else**
- 18 Update $V \leftarrow V \setminus \{j\}$;
- 19 **return** the 1D-chain MPS;

tensors in $A^{(i)}$ to the tail (Line 4) and in $A^{(j)}$ to the head (Line 5), then merges them by contracting edge (i, j) (Line 6). The edge is removed from E (Line 7). For each neighbor k of j (Line 8), edge (j, k) is removed (Line 9). If k connects to i (Line 10), duplicate edges (i, k) are swapped to be adjacent (Line 11) and merged, increasing $D_{k, i}$ (Line 12). If k is not

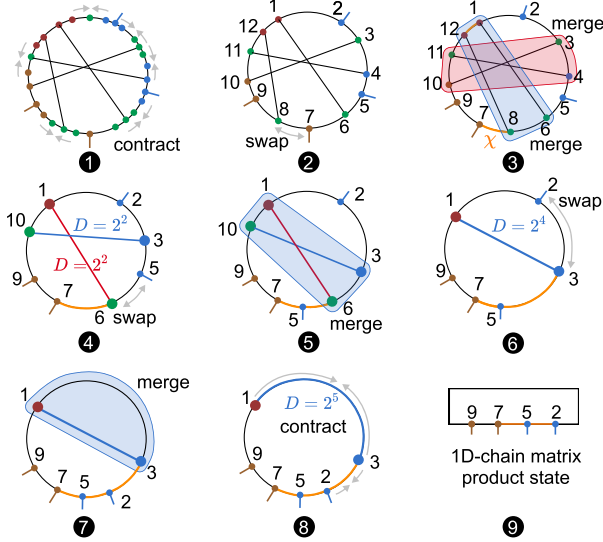


Fig. 7. Offline contraction to get the approximated matrix product state for free energies under different syndromes.

connected to i , edge (i, k) is added to E (Line 14). If $j \in P$, i is added to P (Line 16); otherwise, j is removed from V (Line 18). The final 1D MPS chain is returned (Line 19).

Figure 7 gives an example of the *swap-merge-contraction* approach. We start by *contract* the edges in the MPS representation with the lowest contraction costs as shown in step ①. Then we *swap* the two MPS in edge (8,7) as step ② to make the two pairs of edges (12,8) and (1,6) in adjacent positions as step ③. We *merge* the two pairs of edges $\{(12,8) \text{ and } (1,6), (11,4) \text{ and } (10,3)\}$ into two new edges with the bond dimension $D = 2^2$ as shown in step ④. The steps ④-⑦ also repeat this *swap* and *merge* process until no edges are inside the circle. Then, we *contract* the matrices (1,3) on the circle as in step ⑧, except for the tensors $\{9,7,5,2\}$ with outer logical and syndrome indices, to obtain a 1D-chain MPS with only logical and syndrome indices, as shown in step ⑨.

IV. TENSORMLD DECODING FLOW AND ARCHITECTURE

After offline compression, using the 1D-chain matrix product state for decoding still cannot enumerate all logical spins because this direct search is exponential in the number of logical qubits. Thus, we develop a partial tracing strategy to compute the conditional marginal probability of each logical spin in parallel. This strategy takes $O(M\chi^2 + \log K \cdot K^2\chi^4)$ with the number of syndrome checks M , the number of logical qubits K , and the virtual bond χ (the dimension of the matrix) of matrix product states. Based on this bounded online decoding dataflow, we design a hardware-software co-designed acceleration architecture and implement it on the FPGA and GPU devices.

A. Computational Dataflow

To avoid enumerating all logical spins, we propose to calculate the marginal probability of each logical spin. The

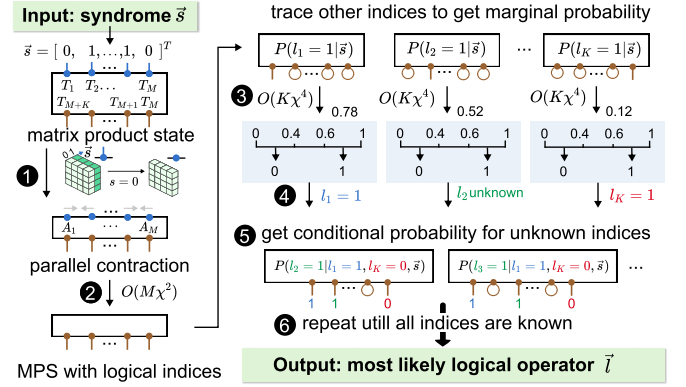


Fig. 8. Online decoding dataflow via marginal probability calculation.

marginal probability is computed as

$$P(l_i = 1|\vec{s}) = \frac{\text{contract}(\mathcal{T}(l_i = 1, \vec{s}))}{\text{contract}(\mathcal{T}(l_i = 1, \vec{s})) + \text{contract}(\mathcal{T}(l_i = 0, \vec{s}))}. \quad (18)$$

This means we only need to contract the 1D-chain MPS twice under the values of g_i to get the marginal probability. However, using the marginal probability may not determine the value of logical spin g_i . For example, when the marginal probability is within the uncertainty interval $[0.4, 0.6]$, the value of logical spin can be 1 or -1 with nearly the same probability. To find the most likely logical spin values, we further calculate their conditional probability under the determined spin values.

Decoding dataflow. Figure 8 illustrates the proposed online decoding flow. The decoder takes the measured syndrome vector $\vec{s}$ as input, transforming the tensors $T_1 \dots T_M$ with syndrome indices (blue legs in Figure 8) in compressed MPS into a sequence of matrices $A_1 \dots A_M$ by fixing one dimension of these tensors based on the observed syndrome s_i , as shown in step ①. Since these inner matrices $A_1 \dots A_M$ have no physical indices, we can merge them by matrix multiplication in parallel with binary tree order in step ②. The resulting MPS then contains only logical indices. Subsequently, the data flow enters a high-throughput computation step ③, where marginal probabilities $P(l_i = 1|\vec{s})$ are inferred in parallel for all K logical qubits. This step follows a parallel pattern, where the MPS chain is duplicated for each logical candidate $\lambda \in \{+1, -1\}$ for parallel computation. These resulting MPS are reduced to scalars ψ_λ using tensor network contraction on other logical indices as $\psi_\lambda = \text{contract}(\mathcal{T}(l_i = \lambda, \vec{s}))$. The marginal probability $P(l_i = 1|\vec{s})$ is then computed by Equation (18). As shown in step ④, if this probability is in the uncertainty interval $[0.4, 0.6]$, the logical operator l_i is labeled as "unknown"; otherwise, the logical operator is fixed as 1 or 0. For the unknown logical operator, the data flow enters an iterative refinement loop as illustrated in step ⑤. In this stage, the MPS is pruned by setting the logical indices with determined logical operators, and marginal probabilities for unknown $P(l_i = 1|\vec{s})$ are recalculated.

Complexity analysis of online decoding. In step ① and step ②, the contraction of the MPS has a cost of $O(M\chi^2)$, where M is the syndrome length and χ the maximum bond

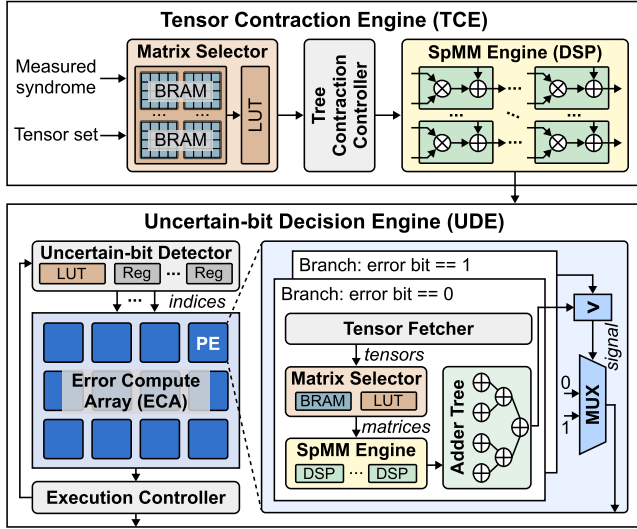


Fig. 9. FPGA architecture support for TensorMLD online decoding.

dimension. The parallel marginal probability calculation in step ③ requires $O(K\chi^4)$ computation, where K is the number of logical spins. The total iteration needs $O(\log K)$ times. Finally, the recovered logical operator $\tilde{l}$ is used to correct the physical state via the Equation (5). For fixed χ , the online cost is $O(M\chi^2 + \log K \cdot K^2\chi^4 + K\chi^4)$ sequentially and $O(M\chi^2 + \log K \cdot K\chi^4 + \chi^4)$ in parallel.

B. GPU Implementation

We implement the online decoding of TensorMLD in CUDA C++ on NVIDIA A100 GPUs. For step ①, we assign one thread block to each site, and SIMT threads in the block generate matrix elements in parallel from the syndrome s_i . In step ②, we organize warps to cooperate on FP64 contractions, and we place intermediate vectors in shared memory to reduce global-memory traffic. We also organize matrix data in GPU memory to keep accesses coalesced and improve reuse of L2 cache and read-only caches.

For the high-throughput step ③ that infers $P(l_i = 1|\vec{s})$ in parallel for all K logical qubits, we launch one thread block per logical qubit, evaluate both logical candidates $\lambda \in \{+1, -1\}$, reduce the two contraction results to ψ_λ , and then compute the marginal probability by Equation (18). We use warp-level shuffle reduction to lower synchronization overhead, and we use two CUDA streams with CUDA events so independent contraction work can overlap while dependencies remain correct. For the iterative refinement loop in step ⑤, we keep unknown-index states on the device, prune the MPS by fixing determined logical indices, and recalculate marginals only for affected regions. This design reduces host-device transfers and lowers per-round decoding latency.

C. FPGA Implementation

Figure 9 shows our FPGA architecture for online decoding of TensorMLD, including a tensor contraction engine (TCE) and an uncertain-bit decision engine (UDE). In the TCE, the matrix selector receives the measured syndrome and

TABLE I
DECODER TECHNIQUES AND SETTINGS. THE PLATFORM COLUMN LISTS THE DECODING HARDWARE THEY CLAIM TO SUPPORT.

Decoder	Goal	Setting	Compute	Platform
BP [35]	MWD	max n iters	Sparse messages	CPU/FPGA
BP+OSD [45]	MWD	OSD order 7	BP + linear solve	CPU
BP+LSD [24]	MWD	30 iters	Local linear solve	CPU
Spin-Glass [32]	MWD	Default	Neural predecoder	CPU
Vegapunk [62]	MWD	Greedy iter 3	Greedy decoupling	FPGA
RelayBP [36]	MWD	80/100/60	BP ensembles	GPU/FPGA
Tesseract [5]	MWD	Default	A* search	CPU
NVIDIA Ising [12]	MWD	Default	Neural predecode	GPU
CUDAQX TN [4]	MLD	Default	cuTensorNet	GPU
TensorMLD	MLD	FP64	1D-chain MPS	GPU/FPGA

tensor set, uses LUT-based index mapping, and reads tensor slices from BRAM to form syndrome-conditioned matrices $A_1, \dots, A_M$. The tree contraction controller then schedules contractions in binary-tree order and dispatches them to the DSP-based SpMM Engine. This stage implements the matrix-merging step in the decoding flow and outputs a contracted state that contains only logical indices, which is forwarded to the UDE. The UDE implements the high-throughput marginal computation and iterative refinement loop. The uncertain-bit detector checks the current marginals and marks logical indices in the uncertainty interval as unknown, then issues these indices to the error compute array (ECA). Under the execution controller, each PE in the ECA evaluates both branches (error bit == 0 and error bit == 1): tensor fetcher loads branch data, matrix selector generates branch matrices, SpMM engine performs contractions, and the adder tree reduces results to two likelihood scalars. A comparator produces the decision signal, and the MUX selects the output bit. After each round, the controller fixes resolved logical indices, prunes the MPS state, and re-launches only unresolved indices until no unknown logical bit remains.

V. EVALUATION

A. Evaluation Setup

Metrics. We quantify the effectiveness of TensorMLD through three critical performance metrics: (1) the logical error rate (LER), (2) the decoding latency, and (3) tensor compression. Upon accumulating d rounds of syndromes (where d is the code distance), TensorMLD applies a correction operation to the logical qubit and subsequently measures its state. A logical error happens if this final logical measurement diverges from the pre-initialized state. The decoding process is iteratively executed numerous times under a defined noise model, with each syndrome generated using the Stim package [20]. The LER, denoted as p_l , represents the proportion of

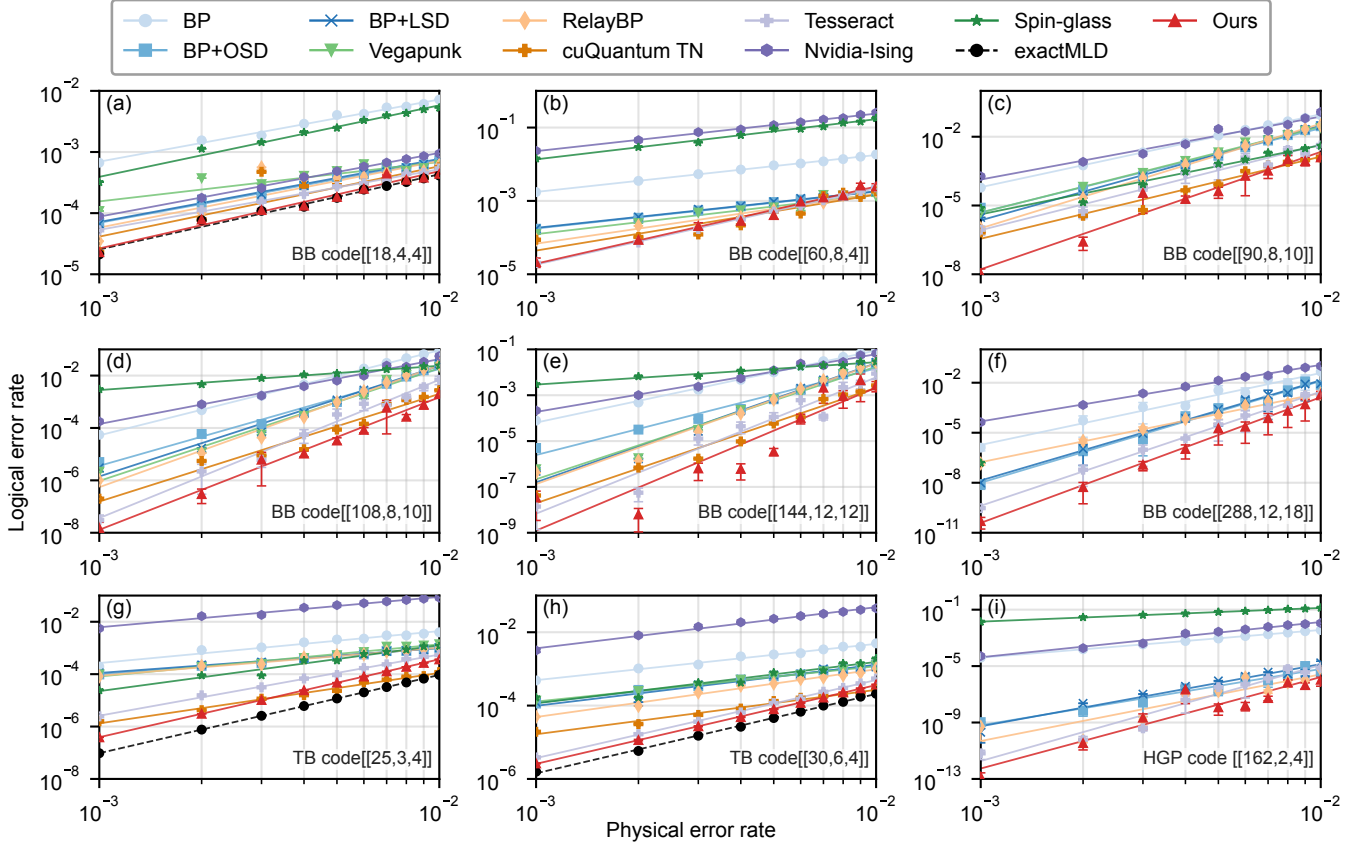


Fig. 10. The logical error rates (LER) for BP [35], BP+OSD [45], BP+LSD [24], Vegapunk [62], RelayBP [36], Tesseract [5], NVIDIA Ising decoder [12], cuQuantum TensorNetwork [4], TensorMLD, and Exact MLD, respectively. The physical error rate p ranges from 1×10^{-3} to 1×10^{-2} . We evaluate the LER using a depolarizing noise model. The LER of exactMLD lacks in other codes because the peak memory requirements of exactMLD have exceeded the total memory of 1TB. The error bars represent confidence intervals from 10 independent repeats with each sampling until 100 logical fails.

decoding failures that result in a logical error relative to the total number of trials.

Baselines. Table I lists the baseline setting used for each decoder. We evaluate the decoding performance of TensorMLD against state-of-the-art decoders for qLDPC codes, including belief propagation (BP) [35], BP with ordered statistic decoding (BP+OSD) [45], BP with localized statistics decoding (BP+LSD) [24], Spin-glass [32], Vegapunk [62], RelayBP [36], Tesseract [5], NVIDIA Ising decoder [12], and NVIDIA CUDAQX Tensor Network Decoder [4]. For BP, we adopt the min-sum update rule and set the maximum number of BP iterations to the number of data qubits n in qLDPC codes to ensure convergence. We use the official LDPC Python library [44] with default settings. For BP+LSD, we set the maximum number of BP iterations to 30 and the OSD order to 0. We use the sequential version of BP+LSD [44], since no open-source parallel implementation is currently available. For Vegapunk, we set the maximum iteration of the greedy decoding algorithm to 3, since it offers the best tradeoff between decoding accuracy and latency. For RelayBP, we set the maximum BP iterations for the first ensemble to 80, the number of relay ensemble elements to 100, and the maximum iterations per relay ensemble to 60. For Tesseract [5], we use

the default open-source setting. For CUDAQX Tensor Network Decoder [4], we use the default configurations, which use CuTensorNet as contractor. Since it only supports one logical qubit, we modify the TN structure to apply it to multiple logical observables.

From a hardware co-design perspective, BP expose sparse message-passing updates that map well to CPU and streaming FPGA. BP+OSD and BP+LSD relies on linear solves which require dynamic pivots and irregular memory access, so we evaluate them on CPU. Vegapunk and RelayBP are the FPGA-oriented MWD baselines: Vegapunk decouples checks into a fixed sparse greedy pipeline, while RelayBP maps many independent BP ensembles to GPU/FPGA parallelism [36]. Tesseract’s search-based MWD has irregular priority-queue control, NVIDIA Ising uses a GPU neural predecoder for an Ising/MWD objective, and CUDAQX TN relies on cuTensorNet’s GPU contraction optimizer.

Implementation. For the tensor network, we use Quimb 1.11.1 [22] with Python 3.12.1 to evaluate the memory and computation cost of contracting the tensor network. We also use it to implement the offline contraction of the matrix product states. For the online decoding with a 1D-chain matrix product states, we implement it with CUDA 12.0 and leverage cuTensorNet 2.8 for contraction operations. All experiments

TABLE II
QLDPC CODE PARITY-CHECK STRUCTURE, OFFLINE COMPRESSION
RUNTIME, AND MEMORY USAGE. THE ACCUMULATED TRUNCATION
ERROR OF THE COMPRESSED CHAIN IS BOUNDED WITHIN $\epsilon \leq 0.1\%$.

Code	Degree	4-cycle	$\chi_{\epsilon \leq 0.1\%}$	Time(s)	Memory (GB)
BB $[[18, 4, 4]]$	5.25	201	8	0.31	0.20
BB $[[60, 8, 4]]$	5.57	254	> 16	13.81	0.58
BB $[[72, 12, 6]]$	5.45	225	> 16	17.07	0.47
BB $[[90, 8, 10]]$	5.72	339	> 32	41.75	0.93
BB $[[108, 8, 10]]$	5.77	417	> 32	152.75	1.11
BB $[[144, 12, 12]]$	5.74	547	> 64	288.24	1.38
TB $[[25, 3, 4]]$	11.83	4,391	> 64	15.56	0.77
TB $[[30, 6, 4]]$	12.56	6,104	> 64	8.93	0.48
HGP $[[162, 2, 4]]$	4.00	324	> 8	43.16	0.22

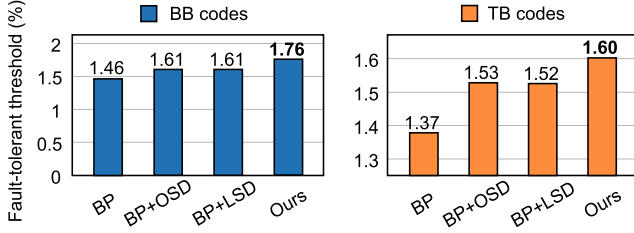


Fig. 11. Fault-tolerant thresholds of BP [35], BP+OSD [45], BP+LSD [24], and TensorMLD, respectively.

are conducted on an AMD EPYC 9554 64-core CPU, NVIDIA A100 80GB GPU, and Xilinx Alveo U50 FPGA targeting a 250 MHz clock. The U50 FPGA provides abundant on-chip resources, including 872K LUTs, 1743K registers, 5952 DSP slices, 5 MB of BRAM, and 20 MB of URAM, which are sufficient to support our proposed FPGA design.

Benchmarks. To comprehensively evaluate the performance of the decoder, we conduct experiments on nine types of QLDPC codes. We adopt six bivariate bicycle (BB) codes [8], including $[[18, 4, 4]]$, $[[60, 8, 4]]$, $[[72, 12, 6]]$, $[[90, 8, 10]]$, $[[108, 8, 10]]$, and $[[144, 12, 12]]$ (the Gross code), and three trivariate bicycle (TB) codes [50], including $[[25, 3, 4]]$, $[[30, 6, 4]]$, and $[[48, 4, 8]]$. Table II reports direct CSS Tanner-graph structure statistics together with the measured offline compression cost. The bounded- χ compression cost is code dependent. High Tanner degree and many 4-cycles in TB codes create dense local overlap, while sparse HGP codes remove these short local overlaps and thus need less virtual bond dimension.

Noise Model and Simulation Stack. We use the Stim simulator [20] to define the syndrome extraction circuit for each code and to inject errors according to the depolarizing noise model. Concretely, the noise model includes: 1) gate errors: each single-qubit gate suffers a depolarizing channel with probability p , applying a uniformly random Pauli (X , Y , or Z); each two-qubit gate (CNOT) suffers a two-qubit depolarizing channel with probability p , applying one of the 15 non-identity two-qubit Paulis; 2) measurement errors: each measurement outcome is flipped with probability p ; 3) preparation errors: each qubit when reset or preparation is

followed by a depolarizing error with probability p . We use 10 independent repeats and continue each repeat until at least 100 logical errors are observed.

B. Logical Error Rate

As shown in Figure 10, TensorMLD achieves the best or near-best LER across the nine evaluated QLDPC codes, with the advantage becoming more pronounced on larger BB and HGP instances. Using the geometric mean over matched physical-error-rate points, TensorMLD reduces LER by $13.6\times$ over BP+OSD, $8.8\times$ over RelayBP, and $9.7\times$ over Vegapunk. On larger BB codes, the gains over BP+OSD are $33.8\times$, $35.3\times$, $48.5\times$, and $29.3\times$ on BB code $[[90, 8, 10]]$, $[[108, 8, 10]]$, $[[144, 12, 12]]$, and $[[288, 12, 18]]$, respectively. Compared with Tesseract [5], TensorMLD obtains a $2.70\times$ overall geometric-mean reduction, with stronger gains on BB code $[[90, 8, 10]]$, $[[108, 8, 10]]$, $[[144, 12, 12]]$, $[[288, 12, 18]]$, and HGP code $[[162, 2, 4]]$. On BB code $[[18, 4, 4]]$, the LER of TensorMLD is no less than 99% of that of the exact MLD. The accuracy advantage of TensorMLD stems from its algorithmic design. TensorMLD is grounded in MLD inference rather than MWD, improving decoding accuracy under highly degenerate QLDPC error patterns.

Fault-tolerant threshold. We calculate the fault-tolerant threshold of different decoders in different code types using the data in Figure 10. The fault-tolerant thresholds are shown in Figure 11. TensorMLD outperforms other decoders in both BB codes and TB codes with thresholds of 1.76% and 1.60%, respectively. The thresholds of BP+OSD and BP+LSD are similar and lower than TensorMLD by 0.15% and 0.08% in BB codes and TB codes, respectively. BP has the lowest performance, which is lower than TensorMLD by 0.30% and 0.23%. The high fault-tolerant threshold of TensorMLD means that it has a higher fault-tolerance ability compared with other minimum weight decoders. TensorMLD also improves the deployability of QLDPC codes on quantum hardware, since reducing the physical error rate in hardware takes much more effort than designing better codes. Notice that the higher fault-tolerant threshold does not imply higher logical error rates. Around the fault-tolerant threshold, the logical error rate of TensorMLD is still lower than that of other decoders.

C. Decoding Performance

Decoding latency. Figure 12 plots the latency and LER of all evaluated decoders across nine QLDPC codes. We organize the decoders by platform: CPU-based decoders (BP [35], BP+OSD [45], BP+LSD [24], Tesseract [5], NVIDIA-Ising [32]) are evaluated on an AMD EPYC 9554 64-core CPU with 1.5TB memory, GPU-based decoders (RelayBP [36], NVIDIA Ising decoder [12], CUDAQX tensor network [4], TensorMLD) are evaluated on an H100 GPU, and FPGA-based decoders (Vegapunk [62], RelayBP [36], TensorMLD) are compared using the corresponding cycle models. We set the virtual bond dimension of TensorMLD to $\chi = 128$ unless otherwise stated. The timing result remains below the 1 ms reference line for all TensorMLD GPU and

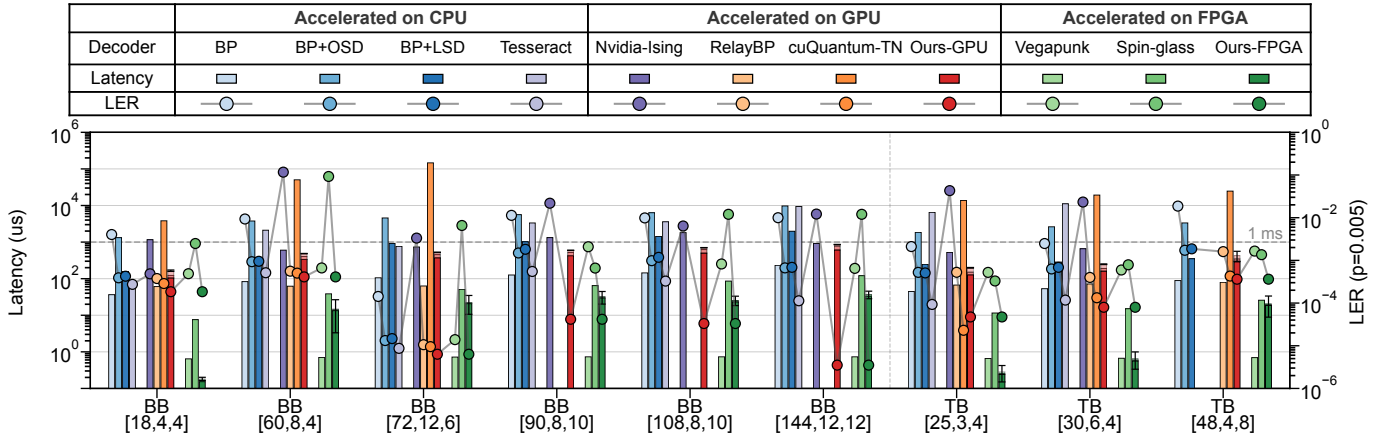


Fig. 12. Decoding latency and LER of BP [35], BP+OSD [45], BP+LSD [24], Tesseract [5], Spin-glass [32], RelayBP [36], cuQuantum TensorNetwork [4], Vegapunk [62], NVIDIA-Ising decoder [12], and TensorMLD. The error bars represent confidence intervals from 10 independent repeats.

TABLE III
FPGA USAGE DETAILS OF TENSORMLD.

Code	LUT	FF	DSP	BRAM
BB [[18,4,4]]	14551(1.67%)	20190(1.16%)	32(0.54%)	11(0.41%)
TB [[25,3,4]]	14180(1.63%)	19983(1.15%)	40(0.67%)	21(0.78%)
TB [[30,6,4]]	14534(1.67%)	20284(1.16%)	45(0.76%)	27(1.00%)
BB [[60,8,4]]	15512(1.78%)	20512(1.18%)	40(0.67%)	380(14.14%)
BB [[90,8,10]]	16133(1.85%)	20593(1.18%)	40(0.67%)	658(24.48%)
BB [[108,8,10]]	17346(1.99%)	20729(1.19%)	40(0.67%)	946(35.19%)
BB [[144,12,12]]	17948(2.06%)	21043(1.21%)	40(0.67%)	1224(45.54%)

FPGA points. At the same time, the LER markers show that this timing is obtained without moving to the higher-error regime of lightweight minimum-weight decoders.

GPU platform. Among the GPU decoders, TensorMLD achieves 165 ~ 856 μs end-to-end latency across all benchmarks, including syndrome input, decoder compute, and physical correction. RelayBP has lower raw latency on several small and TB codes because it uses lightweight message passing, but it also has higher LER on these qLDPC instances. The cuDAQX tensor network decoder, using a general contraction path optimizer, incurs 3.8 ms ~ 146 ms latency on the codes where it completes, which is much slower than the end-to-end latency of TensorMLD. The gap grows with code size because cuDAQX yields high-dimensional, entangled tensors when applied to the qLDPC-specific structure. The compute component of TensorMLD scales from 105 μs to 609 μs , which reflects the linear parallel marginal-probability calculation over logical and syndrome qubits.

FPGA platform. TensorMLD achieves 0.18 ~ 36.9 μs end-to-end latency in the FPGA cycle model. For small codes such as BB [[18,4,4]] and TB [[25,3,4]], it remains sub-microsecond while delivering lower LER than MWD accelerators. For larger codes, the latency grows because the 1D chain is longer and the logical-vector score uses more tensor slices, but it remains within the real-time budget of neutral-atom platforms ($10^2 \sim 10^3 \mu s$). The FPGA prototype of TensorMLD reduces end-to-end decoding latency by over two orders of magni-

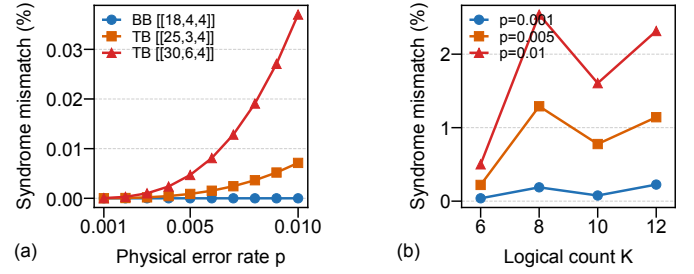


Fig. 13. Per-syndrome mismatch rate between the conditional marginal probability decision and exact MLD. (a) Mismatch rate on qLDPC codes as p changes. (b) Mismatch rate on randomly generated codes as the number of logical qubits increases.

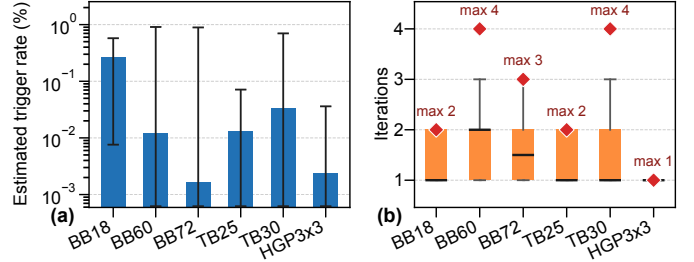


Fig. 14. Refinement trigger and iteration distribution. (a) Estimated trigger frequency. (b) Iteration count distribution and worst observed iteration count.

tude (up to $7,200\times$) relative to the BP+OSD decoder on the general-purpose CPU, while the GPU implementation is $7.8\times \sim 11.4\times$ faster. Table III reports the FPGA resources, which is measure on the xilinx U50 FPGA board. The main resource usage comes from the BRAM, which stores the compressed chain and the syndrome-conditioned matrices. The LUT, FF, and DSP usage are relatively low, which indicates that the design is efficient in terms of logic and arithmetic operations.

D. Decision Diagnostics

Figure 13 reports the mismatch rate of TensorMLD online decoding against exact MLD on exact-feasible cases. The

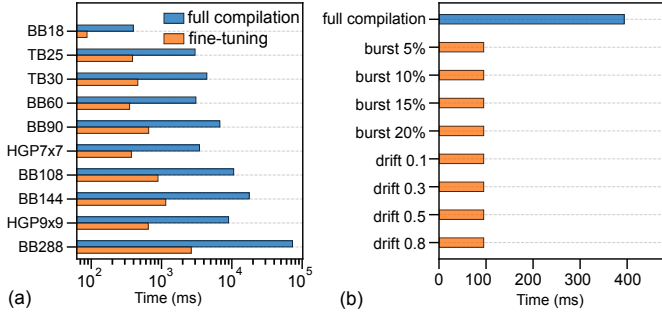


Fig. 15. Compression reuse under transient burst and gradual drift. (a) Full compression time and fine-tuning under different codes. (b) Median time reduction under burst and drift settings on BB $[[18,4,4]]$.

mismatch rate is below 0.001% for BB $[[18,4,4]]$ across $p = 0.001$ to 0.01 , below 0.008% for TB $[[25,3,4]]$, and below 0.038% for TB $[[30,6,4]]$. In other words, the overall decoding accuracy of TensorMLD relative to exact MLD exceeds 99.96%. This is because the number of logical variables is small in qLDPC codes and the logical probability distribution is nontrivial. TensorMLD leverages this to make logical finding more hardware-friendly.

Figure 14 characterizes the variable-latency part of the online decision. In the tested settings, the estimated trigger frequency decreases from 0.27% on BB $[[18,4,4]]$ to 0.012% on BB $[[60,8,4]]$ and 0.0017% on BB $[[72,12,6]]$. The worst observed iteration count is four, so the tail latency is bounded. The low trigger frequency and bounded iteration count suggest that the refinement procedure converges quickly.

E. Offline Compression and Fine-tuning

For noise drift and transient bursts, the contraction order and the 1D-chain layout depend on the code graph and the chosen χ , not on the numerical value of each physical error probability. Therefore, TensorMLD fine-tunes the tensor entries according to the updated physical-error weights. Figure 15 (a) shows that this fine-tuning reduces median update time by 77.9% to 96.4% across BB, TB, and HGP codes. Figure 15 (b) gives that under different burst and drift rate, fine-tuning only requires 22 ~ 35% of the full compression time.

Figure 16 reports offline compile time and peak memory comparison with generic tensor-compression baselines, including cotengra [23] and cuTensorNet [4]. TensorMLD completes the shown BB and TB compression tasks within 0.31 s to 288 s and within 1.41 GB peak RSS. By contrast, cotengra [23] and cuTensorNet [4] paths either hit memory growth or dimension limits on larger qLDPC instances, because they do not use the qLDPC-specific exact decomposition. We also observe that exact MPS decomposition benefits cuTensorNet but not cotengra. The difference arises because cuTensorNet’s GPU contractor benefits from reducing local tensor order, but cotengra’s greedy optimizer needs more cost to handle enlarged graph and increased auxiliary bonds.

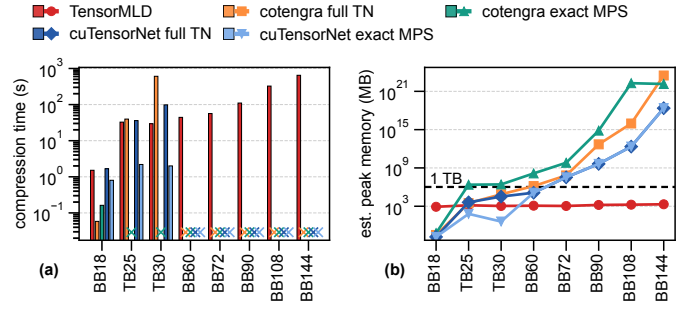


Fig. 16. Offline compression cost and comparison with generic tensor-compression baselines. (a) Compression time. (b) Estimated peak memory.

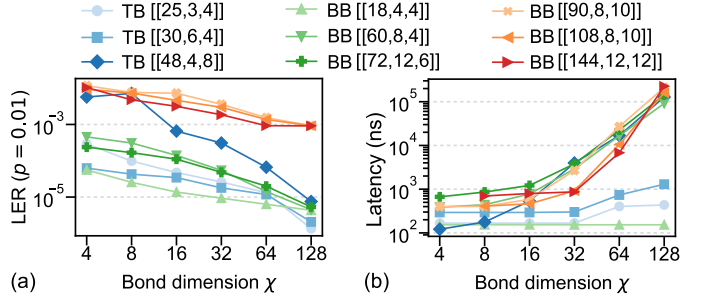


Fig. 17. Bond dimension (χ) sensitivity sweep. (a) LER vs. χ for representative codes at $p = 0.01$. (b) Decoding latency vs. χ .

F. #Swap and Bond Dimension Analysis

Bond Dimension χ Analysis. To show the robustness of our approach, we conduct a χ tolerance sweep as shown in Figure 17. The logical error rate decreases and then saturates at code-specific thresholds. Smaller codes such as BB $[[18,4,4]]$ reach near-exact accuracy with χ as low as 8, while larger codes such as $[[144,12,12]]$ require $\chi \approx 64 \sim 128$ to capture denser stabilizer correlations. Decoding latency and resource use grow with χ , so χ is a tunable accuracy-cost trade-off parameter rather than a hidden constant.

LER vs. bond dimension. Figure 17 (a) plots the logical error rate as a function of bond dimension χ across all benchmark codes at physical error rate $p = 0.01$. The LER decreases monotonically with increasing χ and saturates at code-dependent thresholds. For smaller codes such as BB $[[18,4,4]]$ and TB $[[25,3,4]]$, $\chi = 8$ already approaches near-optimal accuracy ($\text{LER} \sim 10^{-5}$). For the largest codes ($[[90,8,10]]$, $[[144,12,12]]$), $\chi = 64 \sim 128$ is needed for convergence. This saturation behavior is consistent with our exact decomposition: small codes produce MPS chains with inherently low bond dimensions, so truncation is minimal; larger codes with denser stabilizer connectivity require higher χ to capture the inter-site correlations faithfully.

Latency vs. bond dimension. Figure 17 (b) shows the online decoding latency as a function of χ . For small codes like BB $[[18,4,4]]$, the latency remains nearly constant across all χ values, because the natural bond dimension of the compressed MPS is already below χ . For larger codes, the latency grows superlinearly once χ exceeds the saturation point: BB $[[72,12,6]]$ exhibits a $\sim 214\times$ latency increase from

$\chi = 4$ to $\chi = 128$, and TB [[48,4,8]] shows a $\sim 1050\times$ increase. This scaling is consistent with the $O(\chi^2)$ complexity of the syndrome contraction phase and the $O(\chi^4)$ complexity of the marginal probability phase. The steep growth at high χ underscores the importance of our offline compression. By reducing the tensor network to a 1D chain with bounded bond dimension, we keep $\chi \leq 128$ for most codes while maintaining MLD-level accuracy, directly enabling low-latency online decoding.

VI. RELATED WORKS

Here, we review existing decoders, including minimum-weight decoding approaches and maximum-likelihood decoding approaches.

A. Minimum-Weight Decoding Approaches

The objective of MWD is to identify the single most probable error configuration consistent with the syndrome. This equals finding the ground state of an equivalent spin glass system. Although spin glass accelerators can be leveraged, including digital annealers [3], coherent Ising machines [37], and FPGA-based solvers [26], they are easy to trap in local optima, making them suffer from long iteration time and lower accuracy. Belief Propagation (BP) is dedicated to the decoding tasks due to its linear-time complexity and suitability for sparse Tanner graphs [35], [39]. However, quantum degeneracy and short cycles often limit its accuracy [30]. To mitigate this, several BP variants—such as enhanced feedback [53], neural BP [32], and guided decimation (BP+GD) [60]—have been developed. Furthermore, post-processing techniques like Ordered Statistics Decoding (BP+OSD) [40], [45] and Localized Statistics Decoding (BP+LSD) [24] significantly improve accuracy but introduce higher latency due to computationally intensive matrix inversions [48].

Parallel to BP-based decoders, spin glass accelerators—including digital annealers [3], coherent Ising machines [37], and FPGA-based solvers [26]. While these machines are highly efficient at seeking the ground state, they are fundamentally MWD tools and rely on sequential iterations. Besides, these Ising-based solvers are easy to trap in local optima, making them suffer from long iteration time and lower accuracy than post-processing techniques like BP+OSD and BP+LSD.

B. Maximum-Likelihood Decoding Approaches

In contrast to MWD, MLD sums over all error configurations within each logical error and then selects the most likely logical vector [16]. Prior theoretical works on MLD map the decoding transition to spin-glass phase transitions and use random-bond Ising models to analyze error thresholds [13], [17], [29]. Planar decoding [11] targets exact MLD decoding on planar codes like repetition codes. For non-planar codes like qLDPC codes, it inevitably has exponential complexity. These models establish that MLD requires computing the partition function rather than just a single ground state. Tensor networks offer a powerful algebraic framework for computing these

marginals. While TN contraction is efficient for surface codes [4], [9], [18], its application to qLDPC codes is historically limited by the exponential growth of intermediate tensors due to non-local stabilizer interactions. Our work bridges this gap by introducing a qLDPC-oriented compression pipeline.

VII. DISCUSSION

Currently, the GPU implementation targets high-throughput decoding and does not yet meet the tightest real-time feedback requirement of superconducting platforms. Our accelerator design on the 250 MHz FPGA reduces latency to $0.15 \sim 33\mu s$. On the other hand, the GPU latency already satisfies the latency constraints of neutral-atom and ion-trap systems ($\sim 10^2\mu s$) [55]. Given that qLDPC codes require long-range connectivity, future qLDPC hardware is likely to be built on neutral-atom platforms [43]. In this case, our decoder can offer both high precision and real-time performance. The bond dimension χ is the main parameter for control the accuracy-cost trade-off in TensorMLD. Increasing χ lowers LER until code-dependent saturation, but also increases contraction complexity, matrix size, and memory traffic in the GPU/FPGA. Thus, small χ favors latency, resource use, and practical deployment, while larger χ is appropriate when lower LER is worth additional tensor arithmetic. Because the same 1D-chain schedule and accelerator dataflow are preserved across χ , this trade-off provides a scalable deployment space.

Regarding decoding accuracy, we compare TensorMLD with exact MLD only on exact-feasible cases because the memory requirements of exact MLD exceed 1TB on larger codes. On BB code [[18,4,4]], TensorMLD achieves an average logical error rate (3.3×10^{-4}) comparable to that of MLD (3.1×10^{-4}). Higher decoding accuracy can enable higher noise levels in the future design of large-scale quantum chips. Moreover, higher decoding accuracy also helps relax the qubit connectivity requirements, making the chip less sensitive to underlying hardware constraints [11], [13].

VIII. CONCLUSION

In this paper, we propose a tensor-network-based decoder for the most likely logical error in CSS qLDPC codes. By mapping logical error estimation to the free energy of a spin-glass model, we leverage tensor networks for efficient computation. We construct an exact matrix product state representation and then apply bounded- χ offline compression to reduce the network to a 1D chain. After offline compression, the online decoding path has bounded polynomial cost under fixed χ . The online decision uses marginal probabilities to iteratively finding the final logical vector.

ACKNOWLEDGMENT

This work was supported by the Zhejiang Provincial Pioneer (Jianbing) Project (No. 2026C01006), the Zhejiang Provincial Natural Science Foundation of China for Distinguished Young Scholars (No. LR25F020002), and the Fundamental Research Funds for the Central Universities (No. 226-2026-00075).

REFERENCES

- [1] “Constructing quantum codes,” *Frontiers of Computer Science*, vol. 2, pp. 143–146, 2008. [Online]. Available: <https://journal.hep.com.cn/fcs/EN/10.1007/s11704-008-0014-y>
- [2] N. Alavisamani, S. Vittal, R. Ayanzadeh, P. Das, and M. Qureshi, “Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 818–833.
- [3] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, “Physics-inspired optimization for quadratic unconstrained problems using a digital annealer,” *Frontiers in Physics*, vol. 7, p. 48, 2019.
- [4] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, Y.-L. L. Fang, Y. Gao, J. Guan, J. Gunnel, A. Haidar, A. Hehn, M. Hohnerbach, M. Jones, T. Lubowe, D. Lyakh, S. Morino, P. Springer, S. Stanwyck, I. Terentyev, S. Varadhan, J. Wong, and T. Yamaguchi, “cuquantum sdk: A high-performance library for accelerating quantum science,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2023, pp. 1050–1061.
- [5] L. A. Beni, O. Higgott, and N. Shutty, “Tesseract: A search-based decoder for quantum error correction,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.10988>
- [6] E. Berlekamp, R. McEliece, and H. Van Tilborg, “On the inherent intractability of certain coding problems (corresp.),” *IEEE Transactions on Information Theory*, vol. 24, no. 3, pp. 384–386, 2003.
- [7] J. P. Bonilla Ataides, D. K. Tuckett, S. D. Bartlett, S. T. Flammia, and B. J. Brown, “The xzxx surface code,” *Nature communications*, vol. 12, no. 1, p. 2172, 2021.
- [8] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, “High-threshold and low-overhead fault-tolerant quantum memory,” *Nature*, vol. 627, no. 8005, pp. 778–782, 2024.
- [9] S. Bravyi, M. Suchara, and A. Vargo, “Efficient algorithms for maximum likelihood decoding in the surface code,” *Physical Review A*, vol. 90, no. 3, p. 032326, 2014.
- [10] A. R. Calderbank and P. W. Shor, “Good quantum error-correcting codes exist,” *Physical Review A*, vol. 54, no. 2, p. 1098, 1996.
- [11] H. Cao, S. Zhao, D. Feng, Z. Shen, H. Yan, T. Su, W. Sun, H. Xu, F. Pan, H. Yu, and P. Zhang, “Exact decoding of quantum error-correcting codes,” *Physical Review Letters*, vol. 134, no. 19, p. 190603, 2025.
- [12] C. Chamberland, J. Ollé, M. Li, S. Thornton, and I. Baratta, “Fast and accurate ai-based pre-decoders for surface codes,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.12841>
- [13] C. T. Chubb and S. T. Flammia, “Statistical mechanical models for quantum codes with correlated noise,” *Annales de l’Institut Henri Poincaré D*, vol. 8, no. 2, pp. 269–321, 2021.
- [14] P. Das, A. Locharla, and C. Jones, “Lilliput: a lightweight low-latency lookup-table decoder for near-term quantum error correction,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 541–553.
- [15] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, “Afs: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 259–273.
- [16] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, “Topological quantum memory,” *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.
- [17] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, “Topological quantum memory,” *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.
- [18] A. J. Ferris and D. Poulin, “Tensor networks and quantum error correction,” *Physical Review Letters*, vol. 113, no. 3, p. 030501, 2014.
- [19] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 3, p. 032324, 2012.
- [20] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.
- [21] A. Gong, S. Cammerer, and J. M. Renes, “Toward low-latency iterative decoding of qldpc codes under circuit-level noise,” *arXiv preprint arXiv:2403.18901*, 2024.
- [22] J. Gray, “quimb: a python library for quantum information and many-body calculations,” *Journal of Open Source Software*, vol. 3, no. 29, p. 819, 2018.
- [23] J. Gray and S. Kourtis, “Hyper-optimized tensor network contraction,” *Quantum*, vol. 5, p. 410, Mar. 2021. [Online]. Available: <https://doi.org/10.22331/q-2021-03-15-410>
- [24] T. Hillmann, L. Berent, A. O. Quintavalle, J. Eisert, R. Wille, and J. Roffe, “Localized statistics decoding for quantum low-density parity-check codes,” vol. 16, no. 1, p. 8214. [Online]. Available: <https://www.nature.com/articles/s41467-025-63214-7>
- [25] P. Iyer and D. Poulin, “Hardness of decoding quantum stabilizer codes,” *IEEE Transactions on Information Theory*, vol. 61, no. 9, pp. 5209–5223, 2015.
- [26] Z. JADDI, A. AIT MADI, and M. BENBRAHIM, “Ldpc decoding based on statistical mechanics of spin-glasses: A study,” in *2020 1st International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*. IEEE, 2020, pp. 1–5.
- [27] H. Jung, J. Kang, and J. Ha, “Quantum maximum likelihood decoding for linear block codes,” in *2020 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2020, pp. 227–232.
- [28] E. Knill and R. Laflamme, “Theory of quantum error-correcting codes,” *Physical Review A*, vol. 55, no. 2, p. 900, 1997.
- [29] A. A. Kovalev and L. P. Pryadko, “Spin glass reflection of the decoding transition for quantum error correcting codes,” *Quantum Information & Computation*, vol. 15, no. 9–10, pp. 825–852, 2015.
- [30] K.-Y. Kuo and C.-Y. Lai, “Exploiting degeneracy in belief propagation decoding of quantum codes,” *npj Quantum Information*, vol. 8, no. 1, p. 111, 2022.
- [31] R. Laflamme, C. Miquel, J. P. Paz, and W. H. Zurek, “Perfect quantum error correcting code,” *Physical Review Letters*, vol. 77, no. 1, p. 198, 1996.
- [32] Y.-H. Liu and D. Poulin, “Neural belief-propagation decoders for quantum error-correcting codes,” *Physical review letters*, vol. 122, no. 20, p. 200501, 2019.
- [33] L. Ma, M. Fishman, E. M. Stoudenmire, and E. Solomonik, “Approximate contraction of arbitrary tensor networks with a flexible and efficient density matrix algorithm,” *Quantum*, vol. 8, p. 1580, 2024.
- [34] A. S. Maan and A. Paller, “Machine learning message-passing for the scalable decoding of qldpc codes,” *npj Quantum Information*, vol. 11, no. 1, p. 78, 2025.
- [35] D. J. MacKay and R. M. Neal, “Near shannon limit performance of low density parity check codes,” *Electronics letters*, vol. 32, no. 18, pp. 1645–1646, 1996.
- [36] T. Maurer, M. Bühler, M. Kröner, F. Haverkamp, T. Müller, D. Vandeth, and B. R. Johnson, “Real-time decoding of the gross code memory with fpgas,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.21600>
- [37] P. L. McMahon, A. Marandi, Y. Haribara, R. Hamerly, C. Langrock, S. Tamate, T. Inagaki, H. Takesue, S. Utsunomiya, K. Aihara, R. L. Byer, M. M. Fejer, H. Mabuchi, and Y. Yamamoto, “A fully programmable 100-spin coherent ising machine with all-to-all connections,” *Science*, vol. 354, no. 6312, pp. 614–617, 2016. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.aah5178>
- [38] S. Montanero, E. Montanero, and Evenson, *Introduction to tensor network methods*. Springer, 2018.
- [39] T. Müller, T. Alexander, M. E. Beverland, M. Bühler, B. R. Johnson, T. Maurer, and D. Vandeth, “Improved belief propagation is sufficient for real-time decoding of quantum memory,” *arXiv preprint arXiv:2506.01779*, 2025.
- [40] V. Ninkovic, O. Kundacina, D. Vukobratovic, C. Häger, and A. G. i Amat, “Decoding quantum ldpc codes using graph neural networks,” in *GLOBECOM 2024-2024 IEEE Global Communications Conference*. IEEE, 2024, pp. 3479–3484.
- [41] R. Orús, “Tensor networks for complex quantum systems,” *Nature Reviews Physics*, vol. 1, no. 9, pp. 538–550, 2019.
- [42] F. Pan, P. Zhou, S. Li, and P. Zhang, “Contracting arbitrary tensor networks: general approximate algorithm and applications in graphical models and quantum circuit simulations,” *Physical Review Letters*, vol. 125, no. 6, p. 060503, 2020.
- [43] L. Pecorari, S. Jandura, G. K. Brennen, and G. Pupillo, “High-rate quantum ldpc codes for long-range-connected neutral atom registers,” *Nature Communications*, vol. 16, no. 1, p. 1111, 2025.
- [44] J. Roffe, “LDPC: Python tools for low density parity check codes,” 2022. [Online]. Available: <https://pypi.org/project/ldpc/>

- [45] J. Roffe, D. R. White, S. Burton, and E. Campbell, “Decoding across the quantum low-density parity-check code landscape,” *Physical Review Research*, vol. 2, no. 4, p. 043423, 2020.
- [46] N. Sundaresan, T. J. Yoder, Y. Kim, M. Li, E. H. Chen, G. Harper, T. Thorbeck, A. W. Cross, A. D. Córcoles, and M. Takita, “Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders,” *Nature Communications*, vol. 14, no. 1, p. 2852, 2023.
- [47] B. M. Terhal, “Quantum error correction for quantum memories,” *Reviews of Modern Physics*, vol. 87, no. 2, pp. 307–346, 2015.
- [48] J. Valls, F. Garcia-Herrero, N. Raveendran, and B. Vasić, “Syndrome-based min-sum vs osd-0 decoders: Fpga implementation and analysis for quantum ldpc codes,” *IEEE Access*, vol. 9, pp. 138 734–138 743, 2021.
- [49] S. Vittal, P. Das, and M. Qureshi, “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.
- [50] L. Voss, S. J. Xian, T. Haug, and K. Bharti, “Multivariate bicycle codes,” *Physical Review A*, vol. 111, no. 6, p. L060401, 2025.
- [51] K. Wang, Z. Lu, C. Zhang, G. Liu, J. Chen, Y. Wang, Y. Wu, S. Xu, X. Zhu, F. Jin, Y. Gao, Z. Tan, Z. Cui, N. Wang, Y. Zou, A. Zhang, T. Li, F. Shen, J. Zhong, Z. Bao, Z. Zhu, Y. Han, Y. He, J. Shen, H. Wang, J.-N. Yang, Z. Song, J. Deng, H. Dong, Z.-Z. Sun, W. Li, Q. Ye, S. Jiang, Y. Ma, P.-X. Shen, P. Zhang, H. Li, Q. Guo, Z. Wang, C. Song, H. Wang, and D.-L. Deng, “Demonstration of low-overhead quantum error correction codes,” vol. 22, no. 2, pp. 308–314. [Online]. Available: <https://doi.org/10.1038/s41567-025-03157-4>
- [52] W. Wang, Y. Fan, and R. Li, “Optimal binary codes and binary construction of quantum codes,” *Frontiers of Computer Science*, vol. 8, no. 6, pp. 1024–1031, 2014.
- [53] Y.-J. Wang, B. C. Sanders, B.-M. Bai, and X.-M. Wang, “Enhanced feedback iterative decoding of sparse quantum codes,” *IEEE transactions on information theory*, vol. 58, no. 2, pp. 1231–1241, 2012.
- [54] Wikipedia contributors, “Spin Glass — Wikipedia, the free encyclopedia,” https://en.wikipedia.org/wiki/Spin_glass, 2024, [Online; accessed September 21, 2026].
- [55] S. Wolanski and B. Barber, “Introducing ambiguity clustering: An accurate and efficient decoder for qldpc codes,” in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 02, 2024, pp. 402–403.
- [56] Y. Wu, N. Liyanage, and L. Zhong, “Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 639–654.
- [57] Y. Wu and L. Zhong, “Fusion blossom: Fast mwpm decoders for qec,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 928–938.
- [58] Q. Xia, H. Xie, and L. Yang, “Fault-tolerant encoding and decoding method in steane code: comprehensive analysis of maximum threshold,” *Frontiers of Computer Science*, vol. 20, no. 8, p. 2008907, 2026.
- [59] Q. Xu, J. P. Bonilla Ataides, C. A. Pattison, N. Raveendran, D. Bluvstein, J. Wurtz, B. Vasić, M. D. Lukin, L. Jiang, and H. Zhou, “Constant-overhead fault-tolerant quantum computation with reconfigurable atom arrays,” *Nature Physics*, vol. 20, no. 7, pp. 1084–1090, 2024.
- [60] H. Yao, W. A. Laban, C. Häger, A. G. i Amat, and H. D. Pfister, “Belief propagation decoding of quantum ldpc codes with guided decimation,” in *2024 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2024, pp. 2478–2483.
- [61] K. Yin, X. Fang, J. Ruan, H. Zhang, D. Tullsen, A. Sornborger, C. Liu, A. Li, T. Humble, and Y. Ding, “Symbreak: Mitigating quantum degeneracy issues in qldpc code decoders by breaking symmetry,” *arXiv preprint arXiv:2412.02885*, 2024.
- [62] K. Zhou, L. Lu, D. Xiang, C. Tao, A. Wu, J. Leng, F. Liu, M. Chen, and J. Yin, “Vegapunk: Accurate and fast decoding for quantum ldpc codes with online hierarchical algorithm and sparse accelerator,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 719–732.